Réf: 2012/II2/ N° du Sujet:

Ministry of Higher Education and Scientific Research

University of La Manouba

National School of Computer Science



Stage Report of Enterprise Immersion

Subject

Development of a Human-Computer Interface for controlling an LCD display device

Directed by

AOUADI Achraf

Supervised by: Ali Litaiem

Organism : Cynapsys

Adress : Pôle Elgazala des technologies de communication BP 105 Ariana, 2088 -TUNISIE

Phone : +216 71 857 899

Fax : +216 71 858 794

E-mail: info@cynapsys.de

Academic Year: 2011/2012

Signature of the supervisor

Acknowledgements

Upon completion of this work, we wish to express our gratitude to all those who contributed to the development of our project and especially our supervisor Ali Litaiem for his invaluable advice and availability.

Table of contents

General introduction ……. 8

Chapter 1: Overview ……. 10

1.1Context of the internship…… 10

1.2 Presentation of the host organization …… 10

1.2.1 Generalities……..…….10

1.2.2 Organization…… 11

1.2.3 Sites……. 11

1.3 Project Overview…… 11

1.3.1 Project Background……..… 12

1.3.2 Project Presentation1.3.2.1 Human–computer interaction (HCI) …….. 12

1.3.2.1 Human–computer interaction (HCI)……. 12

1.3.2.2 Embedded system……. 12

1.3.2.3 Arcom VIPER …….. 13

1.3.2.4 Embedded Linux……... 14

1.3.2.5 Work goals…… 14

Chapter 2: Requirements analysis and specification…….. 16

2.1 Requirements specification…… 16

2.1.1 Functional needs…….. 16

2.1.2 Non-functional needs……... 17

2.2 Use case diagrams…….. 17

2.3 Sequence diagrams……. 18

Chapter 3: Application Design…….. 20

3.1 Overall design…… 20

3.2 Detailed architecture…….. 21

3.2.1 The main LCD screen……..… 21

3.2.2 The serial connection……... 21

3.2.3 The VIPER board…… 21

3.2.4 The 2x8-line LCD screen…… 22

3.2.5 Detailed view……... 22

3.3 Design of the HCI…….. 23

Chapter 4: Achievement…..…….. 26
4.1 Work environment…….. 26
4.1.1 Hardware environment…… 26
4.1.2 Software environment……..26
4.2 technical choices…… 27
4.2.1 Choice of C language…….. 27
4.2.2 Choice of the SDL library……... 27
4.2.3 Choice of D-bus…….. 28
4.3 Implementation phase…… 28
4.3.1 Connecting the VIPER card…… 29
4.3.1.1: Connection requirements…… 29
4.3.1.2 Connecting the VIPER card…… 29
4.3.2 Test of the application…… 33
4.4 The tests…….. 34
4.5 Documentation……... 35
4.6 Timeline…… 35
General conclusion……..……... 37
Bibliography…… 38
Netography……... 39
Annex A……... 40
Annex B……... 41
Annex C……... 42

List of figures

Figure 1.3.2.3: The VIPER Card…….. 13
Figure2.2: The Main use case diagram ……... 18
Figure 2.3: Sequence diagram of the application……..19
Figure 3.1: The application's overall design…… 20
Figure 3.2.3: the small LCD screen……. 22
Figure 3.2.5: Detailed view of the system……... 22
Figure 3.3: the activity diagram of the HCI……. 24
Figure 4.3.2.1A Main vue of VIPER card……... 30
Figure 4.3.2.1B: the card's redwires…… 31
Figure 4.3.2.1C: th power supply cable……... 31
Figure 4.3.2.1D: the VIPER's main components…… 32
Figure 4.3.2A: Main view of the HCI……. 33
Figure 4.3.2B: Demonstration…….. 34
Figure 4.6: Timeline……. 36

Project Summary & Acronyms

The main purpose of the project is to develop a Human-Computer Interaction to command an LCD display device of an electronic card called VIPER. The HCI must allow its user to send alphanumeric expressions to the small LCD screen embedded in the VIPER card from another external LCD screen connected to the card with a serial cable.

The acronyms used in this report are:

- J2EE: Java 2 Enterprise Edition
- LCD: Liquid Crystal Display
- HCI: Human-Computer Interaction
- AEL: Arcom Embedded Linux
- GUI: Graphical User Interface

General introduction

Each day, our lives become more dependent on 'embedded systems', digital information technology that is embedded in our environment. This includes not only safety-critical applications such as automotive devices and controls, railways, aircraft, aerospace and medical devices, but also communications, 'mobile worlds' and 'e-worlds', the 'smart' home, clothes, factories etc.

All of these have wide-ranging impacts on society, including security, privacy and modes of working and living. More than 98% of processors applied today are in embedded systems, and are no longer visible to the customer as 'computers' in the ordinary sense. New processors and methods of processing, sensors, actuators, communications and infrastructures are 'enablers' for this very pervasive computing. They are in a sense ubiquitous, that is, almost invisible to the user and almost omnipresent. As such, they form the basis for a significant economic push. Within this context is conducted our project which was done in Cynapsys IT Hotspot, an information technology consulting enterprise.

Our project consists in developing a Human-Computer Interaction which allows a user to control and command and LCD display device. Specifically, a small-2x8-line LCD screen which is embedded in an electronic card called VIPER. The work will guarantee the communication between an external tool of the card and an internal one. The main purpose of this project is provides the user the ability to send expressions to the small LCD screen, through his external one.

The first chapter will make the overall presentation of the host organization and the internship project. We will try through this part to expose the project as a general framework and to present some notions which will be helpful in the understanding of the done work.

At the second chapter, we will focus on the specifications of the projects. We will identify the main functionalities that the project must offer, together with the non-functional needs of our application.

Throughout the third chapter, we will detail the design of the application. We describe, first, the overall architecture to adopt. Then we will detail the various modules that comprise the proposed architecture

The final chapter will present the environments used for development our application and present the interfaces describing the features implemented the most important such as the manual execution of orders and the launch of automatic validation tests.

Chapter 1: Overview

Introduction

Throughout this chapter, we will try to put this project in its general framework. We will focus primarily on presenting the body of work. Then we will define the framework and criteria of work to do.

1.1 Context of the internship

This internship is part of the second year of the education of engineers at the National School of Computer Science (ENSI). I was led to make a six-week experience considered as an immersion course in the enterprise. We conducted this course in the IT business Cynapsys Hotspot from June 28 to August 27, 2010.

1.2 Presentation of the host organization

In this section, we describe the host organization of the internship with the company, its organization, production areas and sites, notably in Tunisia.

1.2.1 Generalities

Created in 2001, CYNAPSYS is a dynamic player in the business of consulting and computing engineering services. Their mission is to support our customers and partners in the development of efficient and innovative solutions primarily for telecommunications, finance and industry.

1.2.2 Organization

The team of Cynapsys consists of architects software, project managers, developers, testers and consultants to conduct ICT-functioning of major projects. This expertise allows to offer clients solutions with high added value.

The business organization is structured around four areas:

J2EE cluster: This division handles the installation of systems and applications in the J2EE platform

Embedded Systems cluster: This division deals with the development of applications in the embedded, especially for the automotive field.

.NET cluster: This division handles the installation of systems and applications under the. NET platform.

Android cluster: recently added, this division deals with the development of mobile applications supported by Android.

1.2.3 Sites

The company is of German origin, its first site was established in Munchen. Then it expanded its presence by setting up a second site in France. Through its two subsidiaries, Cynapsys has developed a deep knowledge of the European market.

The site of Tunis, the environment where the internship was conducted, is the site of communication technology Elghazela and has about 115 employees working on support staff, application and design.

1.3 Project Overview

In this section, we present the project, subject of this internship by putting it in its general framework and with interest.

1.3.1 Project Background

This course aimed to develop a human-machine interface for controlling the LCD screen of an electronic card (Arcom Embedded Linux Viper).

1.3.2 Project Presentation

Before describing the work to use, it would be useful to present some concepts and some technologies. This will properly introduce the detailed work.

1.3.2.1 Human–computer interaction (HCI)

It is the study, planning and design of the interaction between people (users) and computers. It is often regarded as the intersection of computer science, behavioral sciences, design and several other fields of study. Interaction between users and computers occurs at the user interface (or simply interface), which includes both software and hardware; for example, characters or objects displayed by software on a personal computer's monitor, input received from users via hardware peripherals such as keyboards and mice, and other user interactions with large-scale computerized systems such as aircraft and power plants. The Association for Computing Machinery defines human-computer interaction as a discipline concerned with the design, evaluation and implementation of interactive computing systems for human use and with the study of major phenomena surrounding them. An important facet of HCI is the securing of user satisfaction.

1.3.2.2 Embedded system

An embedded system is a computer system designed to do one or a few dedicated and/or specific function often with real-time computing constraints. It is embedded as part of a complete device often including hardware and mechanical parts. By contrast, a general-purpose computer, such as a personal computer (PC), is designed to be flexible and to meet a wide range of end-user needs. Embedded systems control many devices in common use today.

1.3.2.3 Arcom VIPER

The VIPER is an ultra-low power PC104 compatible single board computer based on the Marvell® 400MHz PXA255 XScale® RISC processor. The PXA255 is an implementation of the ARM compliant, Marvell XScale microarchitecture combined with a comprehensive set of integrated peripherals including, a flat panel graphics controller, DMA controller, interrupt controller, real time clock and multiple serial ports.



Figure 1.3.2.3: The VIPER Card

The VIPER offers a long list of features making it ideal for power-sensitive embedded communication and multimedia applications. The board has been designed to take advantage of the power saving modes of the PXA255 RISC processor and other onboard peripherals to achieve an incredible 1.9 W maximum power consumption. It also supports a very low power standby mode.

1.3.2.4 Embedded Linux

Embedded Linux is the use of Linux in embedded computer systems such as mobile phones, personal digital assistants, media players, set-top boxes, and other consumer electronics devices, networking equipment, machine control, industrial automation, navigation equipment and medical instruments.

Arcom Embedded Linux (AEL) is a standard Linux distribution produced by Arcom. It is optimized to fit within the on-board Flash of Arcom's range of Single Board Computers (SBCs). AEL is based on the standard Linux kernel and user space tools.

1.3.2.5 Work goals

As an important provider, on an international scale, of embedded system technologies and solutions, Cynapsys IT Hotspot is working on preserving its reputation by continuous innovation in this field.

After the very successful product that the company produced in the embedded system cluster which is an embedded multimedia player for automotive supply. The concerned team has been working to improve the VIPER electronic card in order to expand its capabilities.

The main task that was entrusted to us is to develop a human-computer interaction (HCI) to command the small LCD display device of the card. In fact, this task firstly consist of developing a graphical user interface (GUI) allowing the user to write alphanumeric characters via either the physical or a virtual keyboard. The latter has to be implemented in the interface. Having a-two-line-input area, a part of the graphical user interface allows the user to see what he is writing on real time. Secondly, this GUI will be integrated in a second medium size LCD display device and will be connected to the VIPER card so the user input could be visible on the first small-size LCD display device which will be the target of the two-line input area.

Conclusion

In this chapter we have tried to put this project in its general context with presenting and describing the host institution and with exposing work goals and the tasks to do. In the next chapter, we will begin the analysis of the needs of the application and its specifications.

Chapter 2: Requirements analysis and specification

Introduction

The successful completion of an application depends on understanding and analysis its functionality. In this chapter we describe the functional requirements and non-functional implementation and then we will present the use case diagrams and sequence diagrams

2.1 Requirements specification

In this section of this chapter, we will detail all functional requirements and non-functional ones that must be taken into consideration during the operation of the development of our application.

2.1.1 Functional needs

The application must allow the user to command the LCD display device of the VIPER card. Commands include the fact that the user will be able to send alphanumeric characters to the LCD screen and delete them at any time.

The application must provide to its users the possibility to type characters either via the computer's physical board or the virtual one that is implemented in the human-computer interaction.

The user must be able to see what he is typing in real time in a small input area that is implemented in the human-computer interaction, this area must contain the final input that will be sent to the small LCD screen of the VIPER card.

The virtual keyboard that is implemented in the human-computer interaction must provide the user with both lowercase and uppercase letters.

2.1.2 Non-functional needs

The inclusion of non-functional requirements such as constraints bound to time, safety and efficiency of the operation is delicate as development methods are generally oriented toward modeling the functional needs. We will be interested in what follows in identifying the non-functional needs of our application.

The application must provide a user-friendly interface and ergonomic encompassing all features offered.

Effectiveness: our product must be efficient.

Maintainability: the different modules of the application must be legible and understandable in order to maintain and update them quickly and easily.

Reusability: our tool must allow the addition and editing eventual functionalities optionally.

Genericity: The code of the different modules of the product must be generic for all the team of Cynapsys.

Time response and memory space: the time response of our application must not bother the system. And the memory space occupied by the application should be reduced as much as possible.

2.2 Use case diagrams

In this section of this chapter, we will introduce the different functions of our application. They are to be developed through a use case diagram.

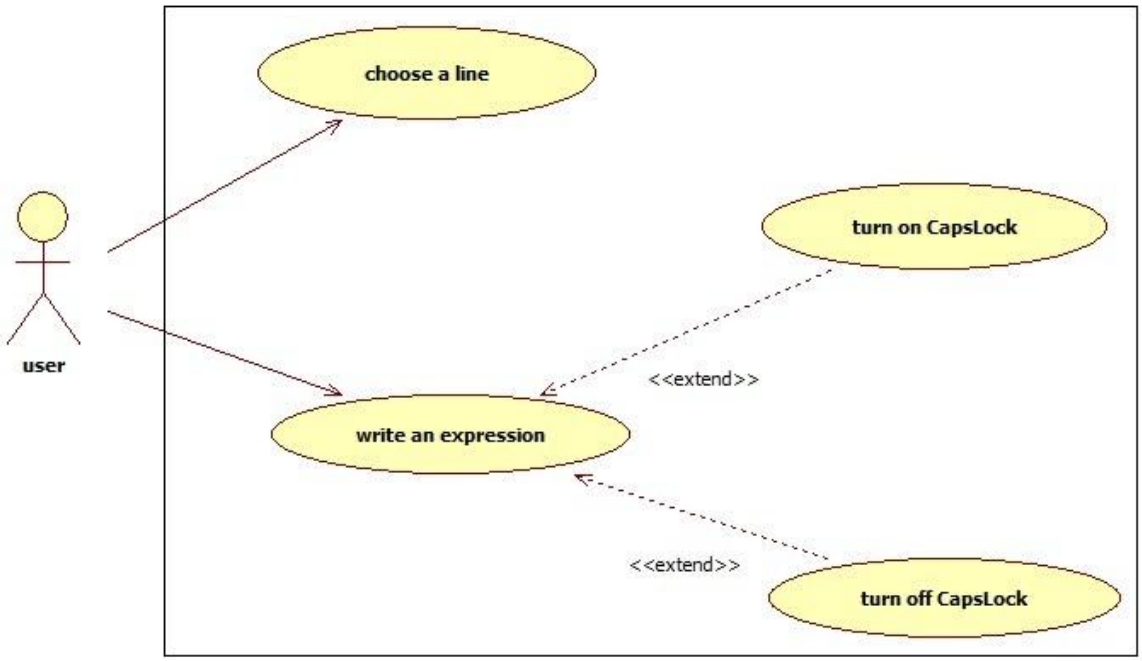


Figure2.2: The Main use case diagram

The user has the possibility to choose a line, by pressing the enter button, between in the 2x8 line LCD screen in which he can write an expression or an operation either with uppercase characters or lowercase ones.

When the second line becomes empty, the cursor automatically switches to the first one.

2.3 Sequence diagrams

In this section of this chapter, we will introduce how a sample scenario works through a typical simple sequence diagram.

As the medium size LCD display device is always connected to the VIPER card, we will not face the phase of the connection to the electronic card. A simple user seeing the human-computer interface can choose to start typing either in the first or the second line of the small LCD screen. The latter is a 2x8 line LCD screen. That means that the latter can display only 8 characters at a time.

The user can intuitively use the CapsLock button in the virtual keyboard to switch between uppercase and lower case characters. Once he presses enter, which means that he finished

typing a line. The input will be immediately transferred to the VIPER card which will display it on its 2x8 line LCD screen. The user can see then what he typed appearing on our target.

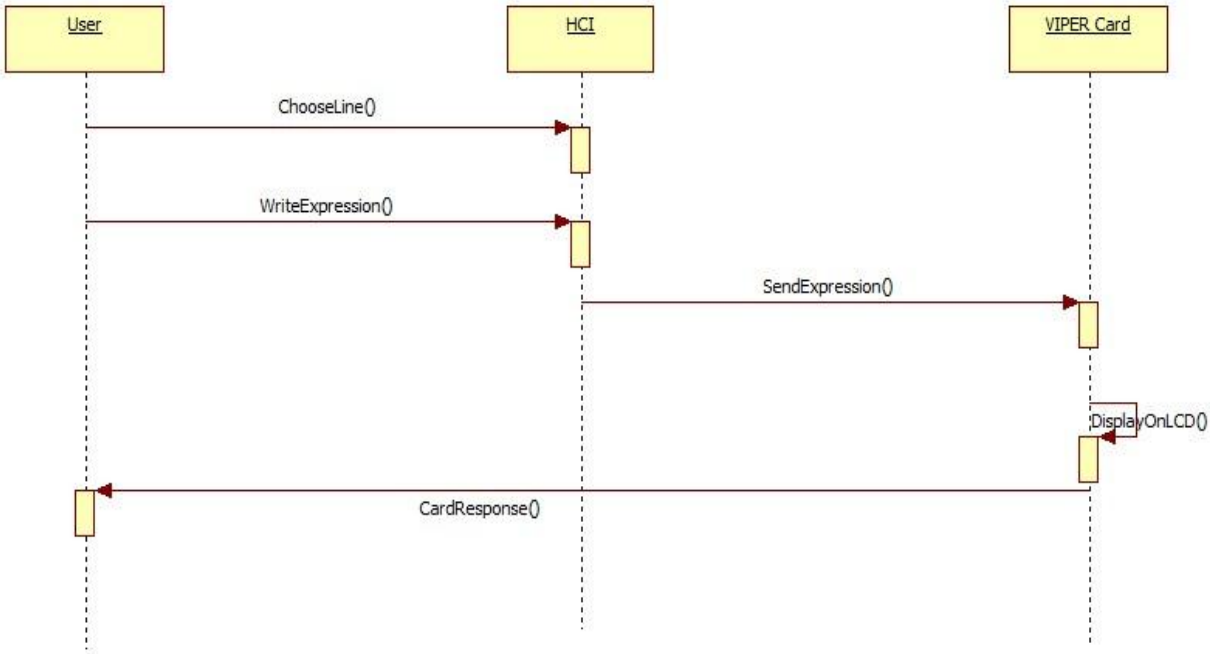


Figure 2.3: Sequence diagram of the application

Conclusion

In this chapter we specified the functional requirements and non-functional requirements of our application. After having presented the use case diagram, we attempted to write and describe the main functions through a sequence diagram.

Chapter 3: Application Design

Introduction

In this chapter, after specifying the various requirements to be met by our application, we present the design of our tool. This chapter focuses on two parts. The first will address the overall design of our work and the second will be devoted to the detailed architecture.

3.1 Overall design

The overall design of our application is represented by Figure 3.1



Figure 3.1: The application's overall design

The figure 3.1 shows the way how the flow of information circulates. In fact, as the main LCD screen has an output area in which the user can see what he writes, the VIPER board can only read what the user has specified via the virtual or the physical keyboard. The application sends the written expression to the main VIPER board via a D-bus connection. The board then analyzes its length and its composition and then displays it on the target LCD screen.

The target LCD screen is directly connected to the VIPER card, so no need to use a D-bus connection to perform sending the information.

3.2 Detailed architecture

In this section, we will have a deeper view on the project architecture to specify more the tools involved in the functioning of the application.

3.2.1 The main LCD screen

It has been integrated by the Cynapsys team and assembled with the VIPER board in order to provide an interface to command the latter. The LCD screen is connected via a serial communication port supervised by a driver developed also by Cynapsys.

It's important to keep in mind that Cynapsys has forbidden us to take a snapshot to the components developed separately from the VIPER board, the main LCD screen is included.

3.2.2 The serial connection

This module contains all functions necessary to ensure communication in between the serial port the main LCD screen and the VIPER board. These functions include the opening and closing the port. They also allow the settings configuration of the connection such as the transmission speed, and the port selection the number of data bits, the type of parity and nature of flow control to use. This module provides also the features necessary for sending and bits reception. It represents then the lowest layer of serial communication with the hardware.

3.2.3 The VIPER board

The board contains many components and interfaces, which are not all involved in the project. It is mainly composed with a PXA255 XScal 400 MHz RISC processor with ARM architecture, a 64-MB of RAM, a 10/100 baseTx Ethernet controller which is connected to the commanding computer through an SSH protocol.

3.2.4 The 2x8-line LCD screen

It is provided and integrated with the VIPER board, its main mission is to show to the user what the board is doing in real time functioning. However it is also used to explain the user what he should do if the board is waiting or needs a human interference.



Figure 3.2.3: the small LCD screen

In fact, the LCD screen can be used in a 16-bit mode. But this is not needed in our application. That's why we turned the 8-bit mode.

3.2.5 Detailed view

After explaining and exposing the main components of our system, we will detail more its functioning.

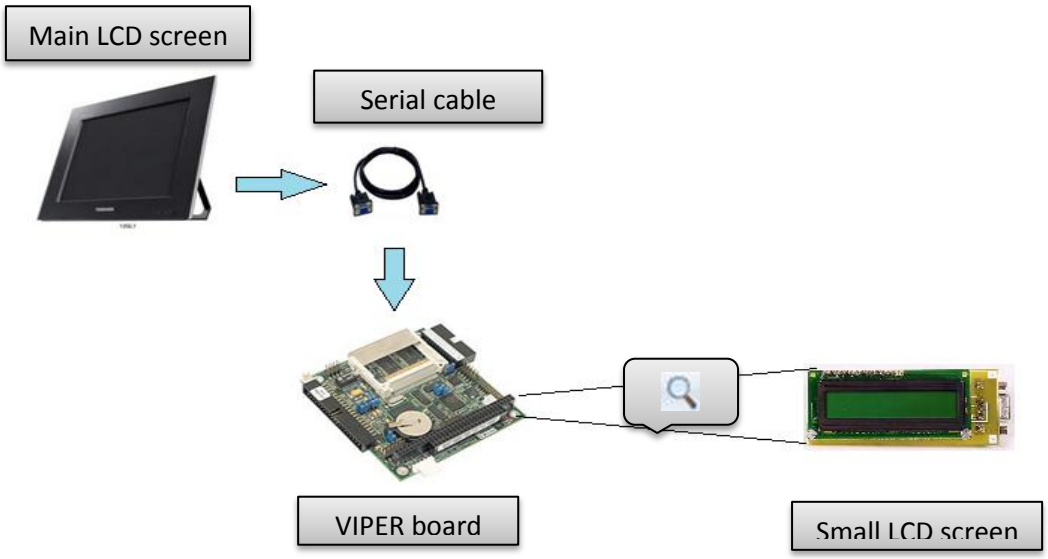


Figure 3.2.5: Detailed view of the system

The figure 3.2.5 shows at first the main LCD screen integrated by Cynapsys team into the whole system. This LCS display device is somehow connected to the VIPER board via a serial cable. A zoom on the board shows the position of the second LCD screen, which is our target.

This figure explains as well how the flow will circulate between our HCI and our LCD screen target.

3.3 Design of the HCI

The activity diagram below explains how a sample process is performed by a simple user of the human-computer interface.

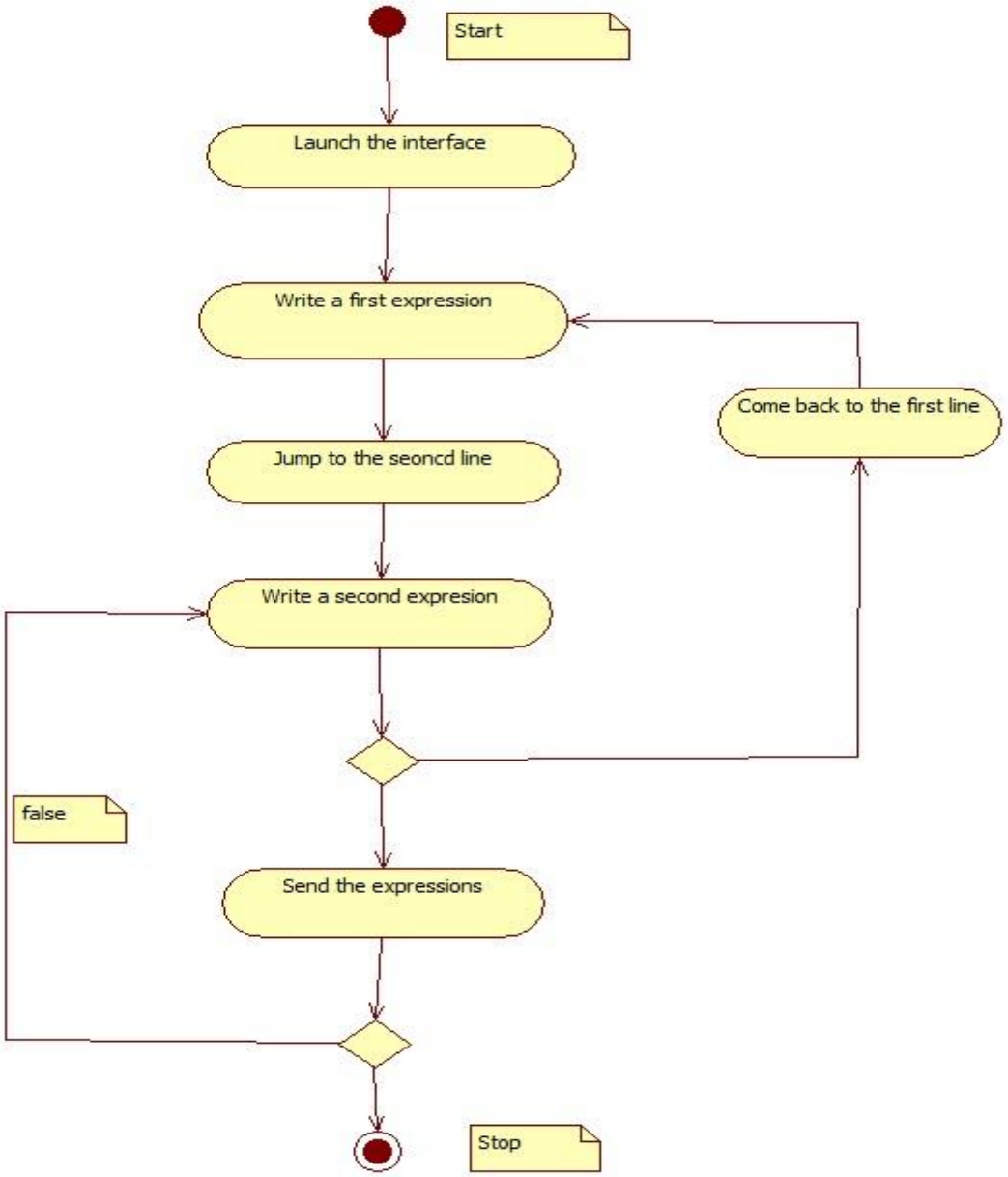


Figure 3.3: the activity diagram of the HCI

Once the interface is launched, the user can start directly righting an expression, when he finishes the first line, he can, at any time, switch to the second line via pressing the “enter” key. The cursor will then switch to the second line where the user has the possibility to write a second expression. The user can only come back to the first line when erasing the totality of the second one. When he finishes writing, and after pressing enter, the message will be sent to the LCD screen and an information message will be displayed to inform the user.

To guide the user in case of failure of sending the expression, it is essential to view warning messages to the failure of an operation represented by the “false” sequence of the diagram. The success of an operation will also be reported a message to the user.

Conclusion:

In this chapter, we presented the design of our project. We all first presented the general architecture of our application. Next, we studied the detailed design of the modules through the UML. This study facilitates the transition to the next stage of implementation. The next chapter is about to highlight the different results of development of the tool required.

Chapter 4: Achievement

Introduction

The latter part of the report is devoted to the operation of the project once the work is completed. We will begin this part by exposure of the hardware and software environment used to develop the application. Then we will present the work and finally, we will present the timeline of the project.

4.1 Work environment

In this part, we will present the hardware and software environment that were used to develop the application.

4.1.1 Hardware environment

To develop our human-computer interface, we used a Toshiba computer which the characteristics are:

Central Processing Unit: Intel Dual Core 2x1.66 GHz

RAM: 2 Go

Cache: 1 Mo

Hard Disk: 250 Go

We had also, as target, the VIPER Card given to us by Cynapsys IT Hotspot

4.1.2 Software environment

To implement our internship project, we used the following software:

Operating system: Ubuntu 11.04

Codeblocks: an Integrated Developping Environment to handle, build and run C code.

VIPER toolchain: an Integrated Developping Environment to build C-written application which the target is the VIPER card.

StarUML: Designing and modeling language.

4.2 technical choices

In this section, we will justify the technical choices of programming language and libraries used.

4.2.1 Choice of C language

In the recent years, the C language has entered into the world of microcontroller. It allows us to take benefit of a universal language and portable, almost independent of the processor used. Compiling with the standard ANSI-C, it is possible to use the same program on any other system (other hardware, other operating system), simply by recompiling.

C language has been developed primarily to program the operating system UNIX, it offers some operations that are close to those of machine language and functions that allow simple and direct access to internal functions of the computer as for example the memory management. It also allows developing rapid and efficient programs.

The extensibility of C language is also an advantage. It does not consist only of standard features, the language is also enriched by libraries or private functions that are delivered by many development houses.

4.2.2 Choice of the SDL library

When implementing this application, we used the SDL library in its version 1.2. The library allows developers to write low-level source code and compile on different platforms such as Windows, Mac and UNIX, therefore ensuring the portability of applications developed with.

Beside, SDL is a free and open-source library and is distinguished by the fact that it is memory and CPU low-consumption. This makes it the ideal library to use when developing embedded system application which has more resource constraints.

4.2.3 Choice of D-bus

In computing, D-Bus (Desktop Bus) is a simple inter-process communication (IPC) open-source system for software applications to communicate with one another. An implementation of D-Bus supports most POSIX operating systems, and a port for Windows exists.

D-Bus allows programs to register with it in order to offer services. It also allows client programs to check for the availability of services. Programs can also register as waiting for events of the kernel, as in the case of hot-swapping hardware.

Currently, D-Bus works over UNIX sockets, but work is being done to create a 'peer-to-peer' socket in the Linux kernel able to route messages between applications, leaving the daemon as a top-level manager. The main advantage of this new approach is that it improves speed by halving the number of memory-copy operations.

Provided that D-Bus is already present in Linux kernel, and above the fact that it is light and low-consuming CPU. It has been chosen as the most convenient communication system between the main LCD screen and the VIPER card.

4.3 Implementation phase

The ergonomics of the Human-Computer interfaces for a given application is among the major criteria for trial of an application. These interfaces should be well designed for a user so he could be comfortable and take advantage from it and use it flexibly.

4.3.1 Connecting the VIPER card

To be able to communicate with the VIPER card, it's necessary to meet some hardware and software requirements.

4.3.1.1: Connection requirements

An x86 Linux system compliant with version 1.3 of the Linux Standard Base2 is required for the Arcom Embedded Linux host environment. Suitable systems include PCs with Debian GNU/Linux, RedHat Linux, Fedora Core or SUSE Linux. The Fedora Core installation DVD is supplied for this purpose.

In addition, the following information and equipment may be useful:

- Networking details for the VIPER. If your network does not have a DHCP server, we should ask for these configurations to be provided

  - IP address and subnet mask (netmask).

  - Default gateway IP address (if required).

  - Name server (DNS) IP address (optional).

- Networking details for the host machine.
- Standard Ethernet cable to connect VIPER / VIPER-Lite to a switch or hub. Alternatively, for standalone systems, the board can be connected directly to your host machine using the crossover cable provided in the Development Kit.

4.3.1.2 Connecting the VIPER card

This section is a guide through the process of connecting up the VIPER card for the first time and powering up the system. The VIPER is delivered ready to run. There are no link settings or configuration switches that need to be adjusted for the board to operate.

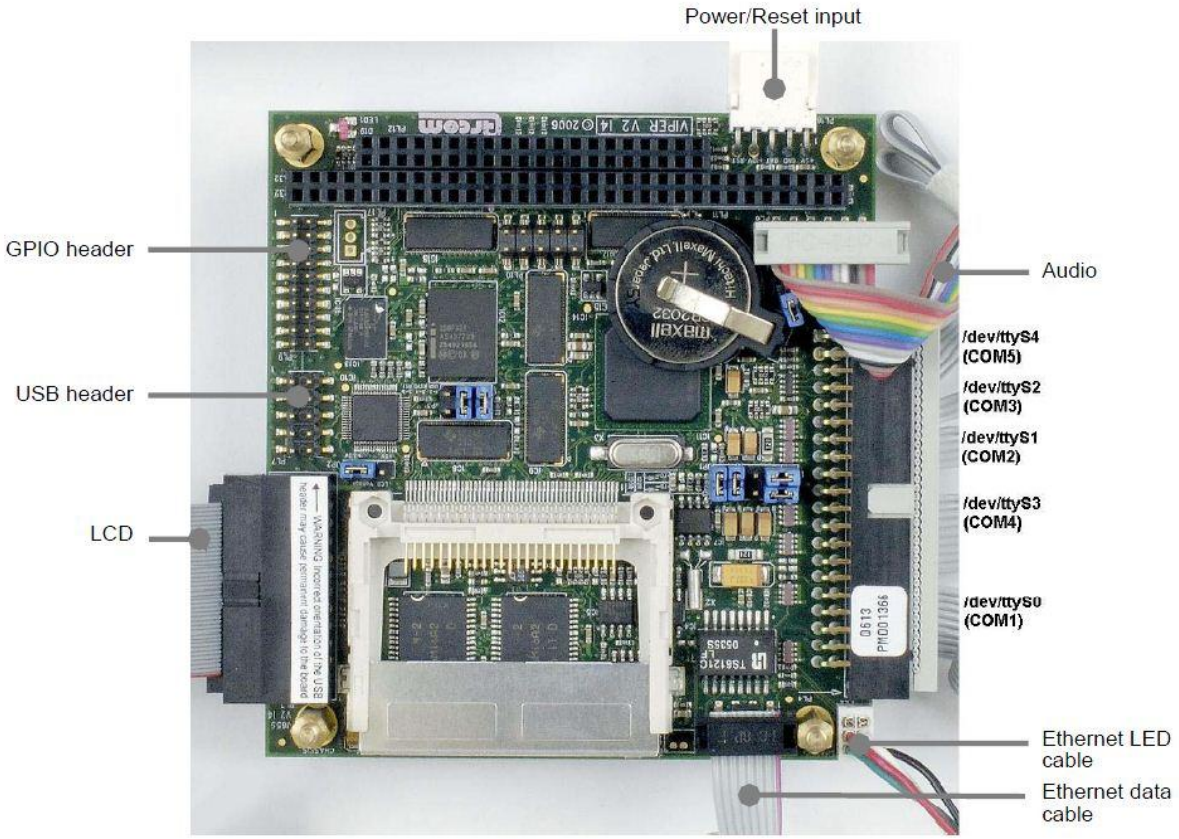


Figure 4.3.2.1A Main vue of VIPER card

The board shown in the figure 4.3.2.1 powers up automatically and loads the RedBoot boot loader from the on-board Flash array. At this point we can interrupt the boot process, if we wish, and access the Redboot command prompt. If we do not interrupt the normal boot sequence, after a 1 second pause, the system continues to load and boot Linux (AEL).

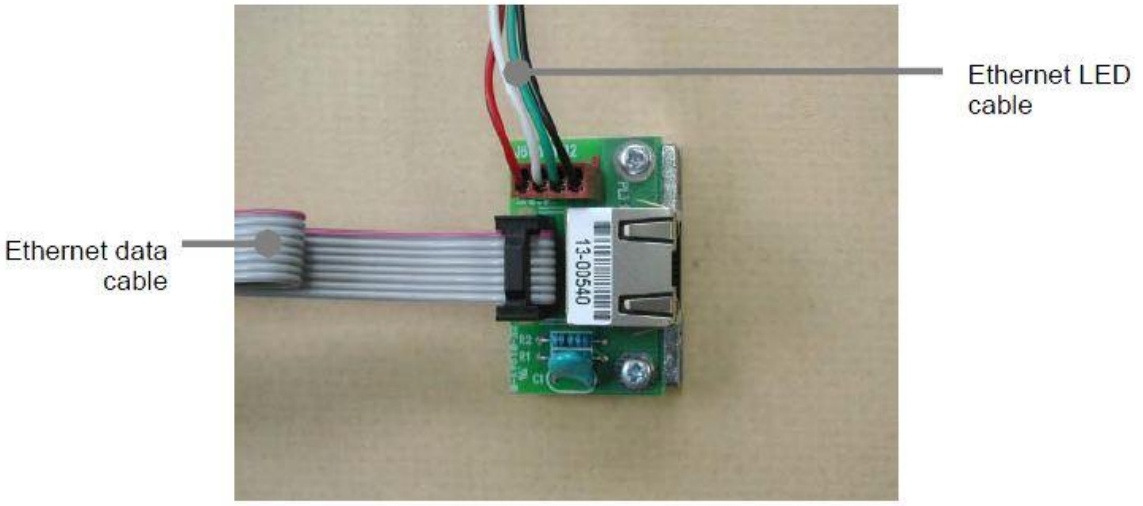


Figure 4.3.2.1B: the card's redwires

In fact, to power up the VIPER card and boot AEL we should firstly know that for all connections, the red wire on the cable shown in figure 4.3.2.1B is pin 1. Pin 1 on each connector is marked on the board. Then the VIPER I/O board can be attached above the main VIPER board once all connections to the main board are complete.

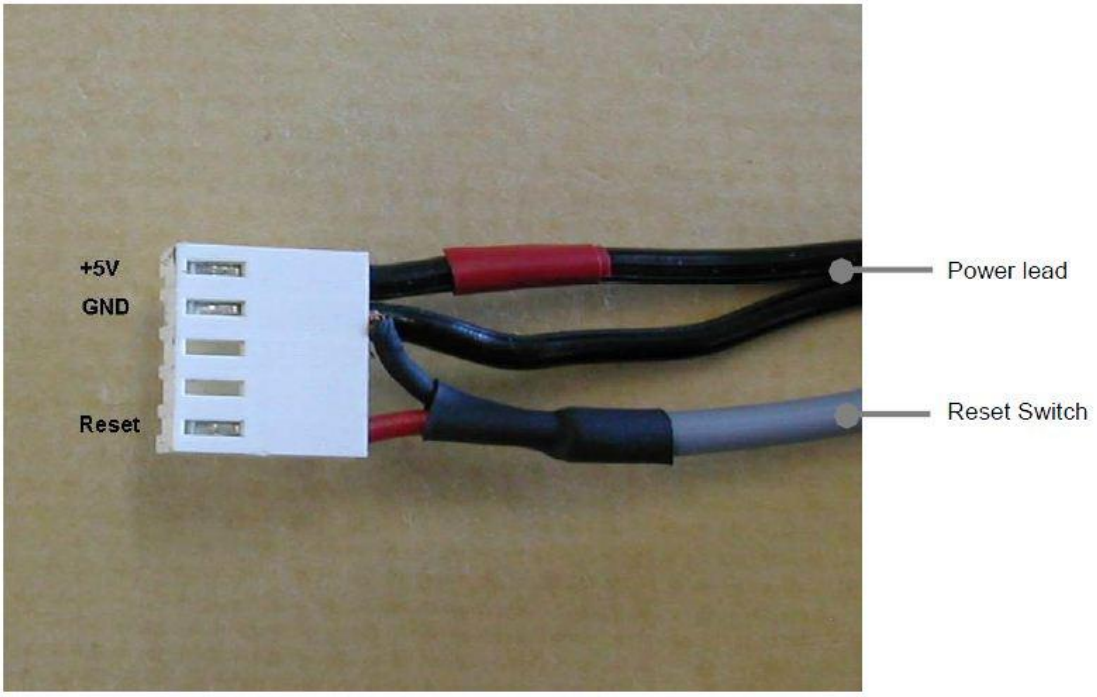


Figure 4.3.2.1C: th power supply cable

Secondly we fit the VIPER power supply cable shown in the figure 4.3.2.1C to the plug on the board. Then we should connect a serial terminal emulator3 to COM1 using the supplied null modem adaptor and cable. After that, we will configure the serial terminal as follows:

- 115200 baud.
- No parity.
- 8 data bits.
- No handshaking.

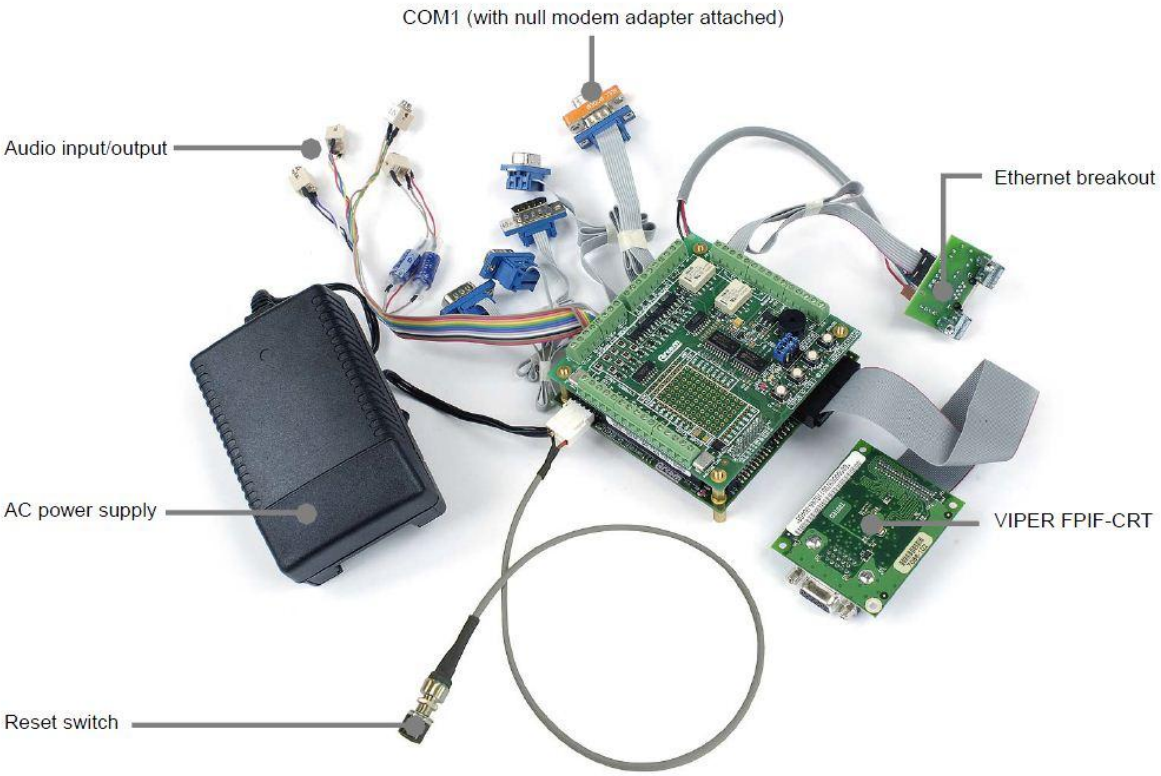


Figure 4.3.2.1D: the VIPER's main components

Finally, once the needed components show in the figure 4.3.2.1D are correctly plugged and configured, we switch on the VIPER board once the terminal emulator is set up on the host PC. The Linux operating system is loaded from the on-board Flash memory, and the board displays a Linux prompt.

4.3.2 Test of the application

After having the VIPER card correctly connected and successfully switched on, and once the network parameters and the main LCD screen wires are correctly set up. We run the application which is implemented in the latter, and we try to write some words via the physical or the virtual keyboard.

As a simple test, we wrote a simple “Hello, World!” expression which is a word that contains both uppercase and lower case characters, and a comma as well. That will test both of the “CapsLock” key and the special characters keys in the keyboards.

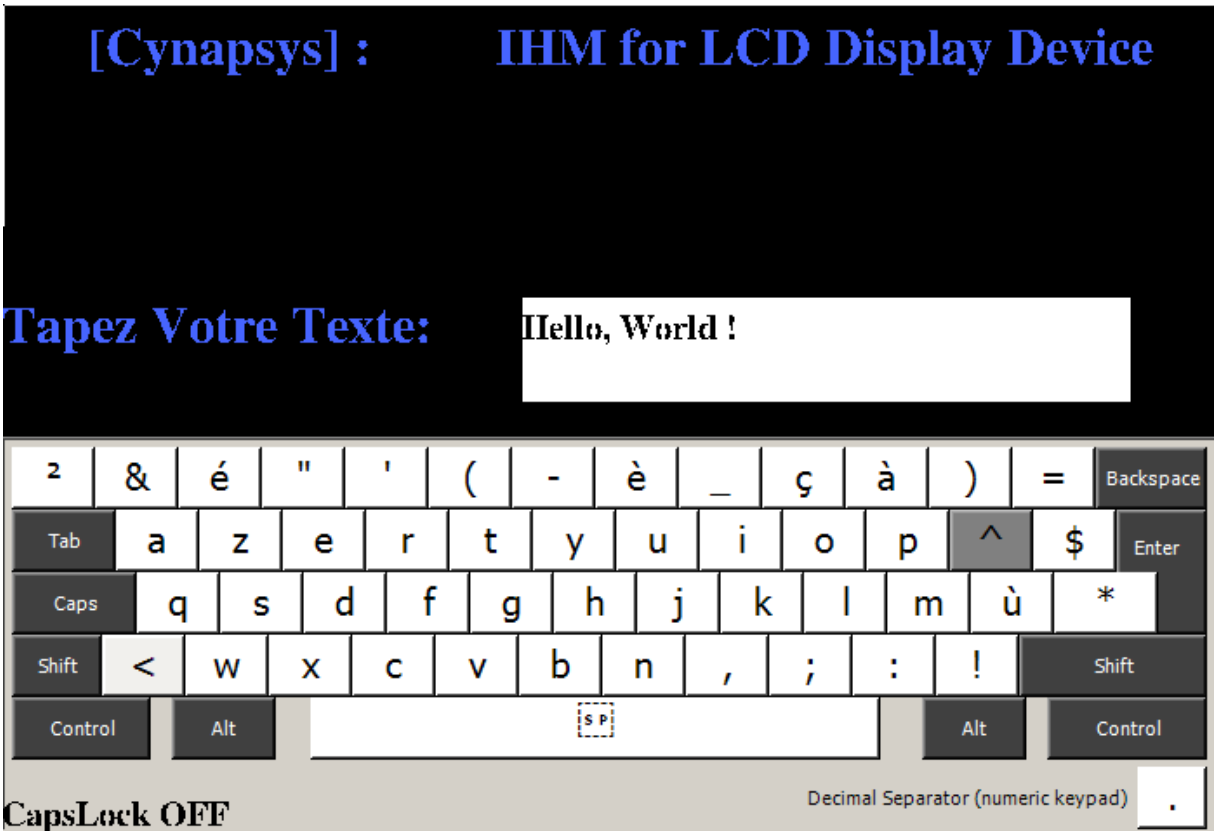


Figure 4.3.2A: Main view of the HCI

After finishing typing and once we type the “Enter” button, the same “Hello, World !” expression will appear in the second LCD screen of the VIPER card as shown in the figure.



Figure 4.3.2B: Demonstration

It’s important to mention that if the word is longer than the LCD screen’s length, it will scroll horizontally and with a cyclic right-to-left way.

4.4 The tests

The tests are an essential part of project monitoring. They are from different types and intervene at several levels of the development process of the application. In fact, they can reveal a number of problems such as:

- Design flaws
- Inefficient procedures
- Insufficient performance
- Incompatibility issues with hardware and / or software

In order to respect the integrity of data and ensure proper operation, it took us to follow and respect all the different stages of testing. All tests must be validated unless it must take them back after making all the necessary corrections.

Here is an overview of the different tests:

Unit testing: Unit tests are to test individual components of the application. It allowed us to validate code quality and performance.

Integration tests: These tests are performed to validate the integration of different modules together and in their final operating environment. they will highlight the problems of interfaces between different programs.

Function tests: These tests are designed to verify the compliance of the application developed with the initial specifications. They are based on functional and technical specifications that have been established at the beginning of the project.

System tests: They correspond to specific tests validating the technical specifications detailed at the beginning of the project.

4.5 Documentation

The writing of the documentation is a continuous and rigorous work throughout the development of project.

Two types of documentation have been used during our project development, namely:

- Source code documentation: which have been produced at the same time of the code writing, it facilitates the understanding of the instructions.
- User documentation: which is oriented for the user, it allows him to understand operation of the tool. In our case, we have provided a small manual leaving the operation of the tool rather clear and easy. The aim is then to make sufficiently clear and intuitive the user interface

4.6 Timeline

This project has been developed from June 27 to August 14, 2011. To try to develop correctly this application, we have followed a sequence of well-defined steps. Some of them were launched in parallel such as documentation of the technologies used and report writing. The aim was to clearly arrange, with a convenient manner, the progress that had been made throughout the project.

We have established a timeline for the different steps of our project achievement, it is presented in the figure 4.6

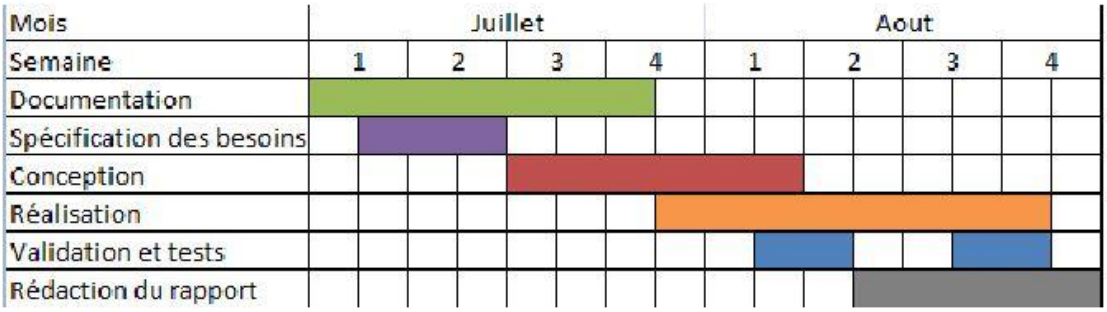


Figure 4.6: Timeline

Conclusion

In this chapter we presented the aspects of the achievement of our application. These aspects have accompanied by printed notches describing and showing the functioning of our application. We have also presented the various tests performed and documentation written in this internship. At the end of this chapter, we described the different steps bearing our project with a timeline. We will now begin the concluding part

General conclusion

Our project goal consisted in developing a Human-Computer Interaction which allows a user to control and command and LCD display device. Specifically, a small-2x8-line LCD screen which is embedded in an electronic card called VIPER. The main purpose of this project is to provide the user the ability to send expressions to the small LCD screen, through his external one.

The development of our work began with a theoretical study phase through which we are introduced to some main embedded systems notions, and to the VIPER card. After the second stage of functional analysis and use cases determination, we have begun the design phase during which we opted for an architectural model and we described the different modules that make up this architecture. Finally, we presented the hardware and software environment that served to frame in the implementation of our application.

Through this project, we were able to learn some aspects of the field of embedded systems and particularly in regard to the operation of LCD screens and the functioning of human-computer interfaces. This project was very useful to learn about serial communication and type conversion in programming. This knowledge will valuable in any future development projects in embedded systems.

Although our application has a complete solution for commanding the VIPER embedded LCD screen and allows the user to send expressions to the latter, it can be enriched by detailing the description of the conduct of tests and trying to further accelerate even more time to send expressions. Such improvements may up the prospects of our project.

Bibliography

[1] Doug Abbot, Linux for Embedded and Real-Time Applications, USA, 2006, 456 pages

[2] Gene Sally, Pro Linux Embedded Systems, 2009, 699 pages

[3] VIPER Entry Level Linux Quickstart Manual, UK, 30 pages

[4] VIPER IO Technical Manual, UK, 19 pages

Netography

[1] http://www.aurel32.net/elec/lcd.php (last consultation: August 20, 2011)

[2] http ://www.beyondlogic.org/serial/serial.htm (last consultation: August 10, 2011)

[3] http://www.eurotech-inc.com/single-board-computer-pxa255-pc104-viper.asp (last consultation: August 8, 2011)

[4] www.siteduzero.com/LibrairieSDL (last consultation: August 5, 2011)

Annex A

Liquid Crystal Display (LCD)

A liquid crystal display (LCD) is a flat panel display, electronic visual display, video display that uses the light modulating properties of liquid crystals (LCs). LCs do not emit light directly.

They are used in a wide range of applications, including computer monitors, television, instrument panels, aircraft cockpit displays, signage, etc. They are common in consumer devices such as video players, gaming devices, clocks, watches, calculators, and telephones. LCDs have displaced cathode ray tube (CRT) displays in most applications. They are usually more compact, lightweight, portable, less expensive, more reliable, and easier on the eyes.[citation needed] They are available in a wider range of screen sizes than CRT and plasma displays, and since they do not use phosphors, they cannot suffer image burn-in.

LCDs are more energy efficient and offer safer disposal than CRTs. Its low electrical power consumption enables it to be used in battery-powered electronic equipment. It is an electronically modulated optical device made up of any number of segments filled with liquid crystals and arrayed in front of a light source (backlight) or reflector to produce images in color or monochrome. The most flexible ones use an array of small pixels. The earliest discovery leading to the development of LCD technology, the discovery of liquid crystals, dates from 1888. By 2008, worldwide sales of televisions with LCD screens had surpassed the sale of CRT units.

Annex B

Human-Computer Interaction

Human–computer interaction (HCI) is the study, planning and design of the interaction between people (users) and computers. It is often regarded as the intersection of computer science, behavioral sciences, design and several other fields of study. Interaction between users and computers occurs at the user interface (or simply interface), which includes both software and hardware; for example, characters or objects displayed by software on a personal computer's monitor, input received from users via hardware peripherals such as keyboards and mice, and other user interactions with large-scale computerized systems such as aircraft and power plants. The Association for Computing Machinery defines human-computer interaction as "a discipline concerned with the design, evaluation and implementation of interactive computing systems for human use and with the study of major phenomena surrounding them." An important facet of HCI is the securing of user satisfaction.

Because human-computer interaction studies a human and a machine in conjunction, it draws from supporting knowledge on both the machine and the human side. On the machine side, techniques in computer graphics, operating systems, programming languages, and development environments are relevant. On the human side, communication theory, graphic and industrial design disciplines, linguistics, social sciences, cognitive psychology, and human factors such as computer user satisfaction are relevant. Engineering and design methods are also relevant. Due to the multidisciplinary nature of HCI, people with different backgrounds contribute to its success. HCI is also sometimes referred to as man–machine interaction (MMI) or computer–human interaction (CHI).

Attention to human-machine interaction is important, because poorly designed human-machine interfaces can lead to many unexpected problems. A classic example of this is the Three Mile Island accident where investigations concluded that the design of the human-machine interface was at least partially responsible for the disaster. Similarly, accidents in aviation have resulted from manufacturers' decisions to use non-standard flight instrument and/or throttle quadrant layouts: even though the new designs were proposed to be superior in regards to basic human-machine interaction, pilots had already ingrained the "standard" layout and thus the conceptually good idea actually had undesirable results.

Annex C

Embedded Linux

Embedded Linux is the use of Linux in embedded computer systems such as mobile phones, personal digital assistants, media players, set-top boxes, and other consumer electronics devices, networking equipment, machine control, industrial automation, navigation equipment and medical instruments. According to survey conducted by Venture Development Corporation, Linux was used by 18% of embedded engineers.

Linux has been ported to a variety of CPUs not primarily used as the processor of a desktop or server computer, including ARM, AVR32, ETRAX CRIS, FR-V, H8300, IP7000, m68k, MIPS, mn10300, SuperH, and Xtensa processors, as an alternative to using a proprietary operating system and toolchain.

The advantages of embedded Linux over proprietary embedded operating systems include no royalties or licensing fees, a stable kernel, a support base that is not restricted to the employees of a single software company, and the ability to modify and redistribute the source code. The disadvantages include a comparatively larger memory footprint (kernel and root filesystem), complexities of user mode and kernel mode memory access, and complex device drivers framework.